



{ یک یادگیری کارساز }

بوتکمپ

Artificial Intelligence

MLOps

Session 6

Welcome & Session Overview

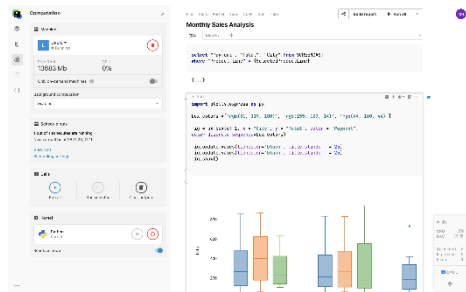
Model Packaging & Deployment Concepts

- Recap: From Model Development to Production Reality
- Today's Journey: Making ML Models Usable
- Learning Objectives:
 - Model serialization and packaging strategies
 - Deployment patterns (batch, real-time, edge)
 - API design for ML models
 - Scalability and performance considerations
 - Security basics for ML applications

The VisionaryAI Challenge

From Notebook to Production

The Success: 95% accuracy defect detection model ✓



The Problem: Model lives in a Jupyter notebook

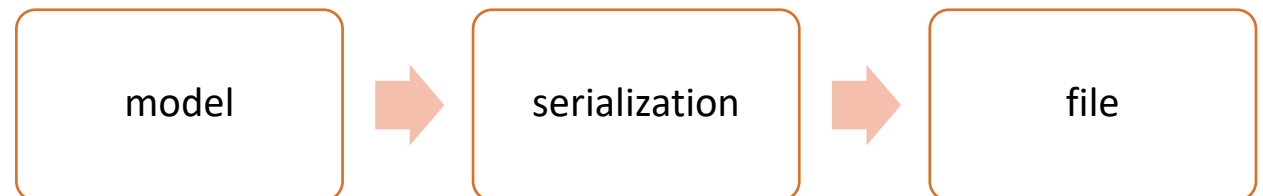


The Question: "How do we use this in the factory?"

What is Model Serialization?

Converting Models for Storage & Transfer

- **Definition:** Converting trained models into storable format
- **Like:** Disassembling a machine, boxing parts with instructions
- **ML Models Contain:**
 - Model architecture (blueprint)
 - Learned parameters (weights, biases)
 - Preprocessing steps
 - Training metadata



Serialization Options Comparison

Popular Serialization Formats

Format	Best For	Pros	Cons
Pickle	Python objects	Simple, universal	Python-only, security risks
Joblib	NumPy/sklearn	Efficient with arrays	Still Python-specific
Framework-specific	Native models	Optimized performance	Framework locked
ONNX	Cross-platform	Framework agnostic	Limited model support

Model Packaging Strategies

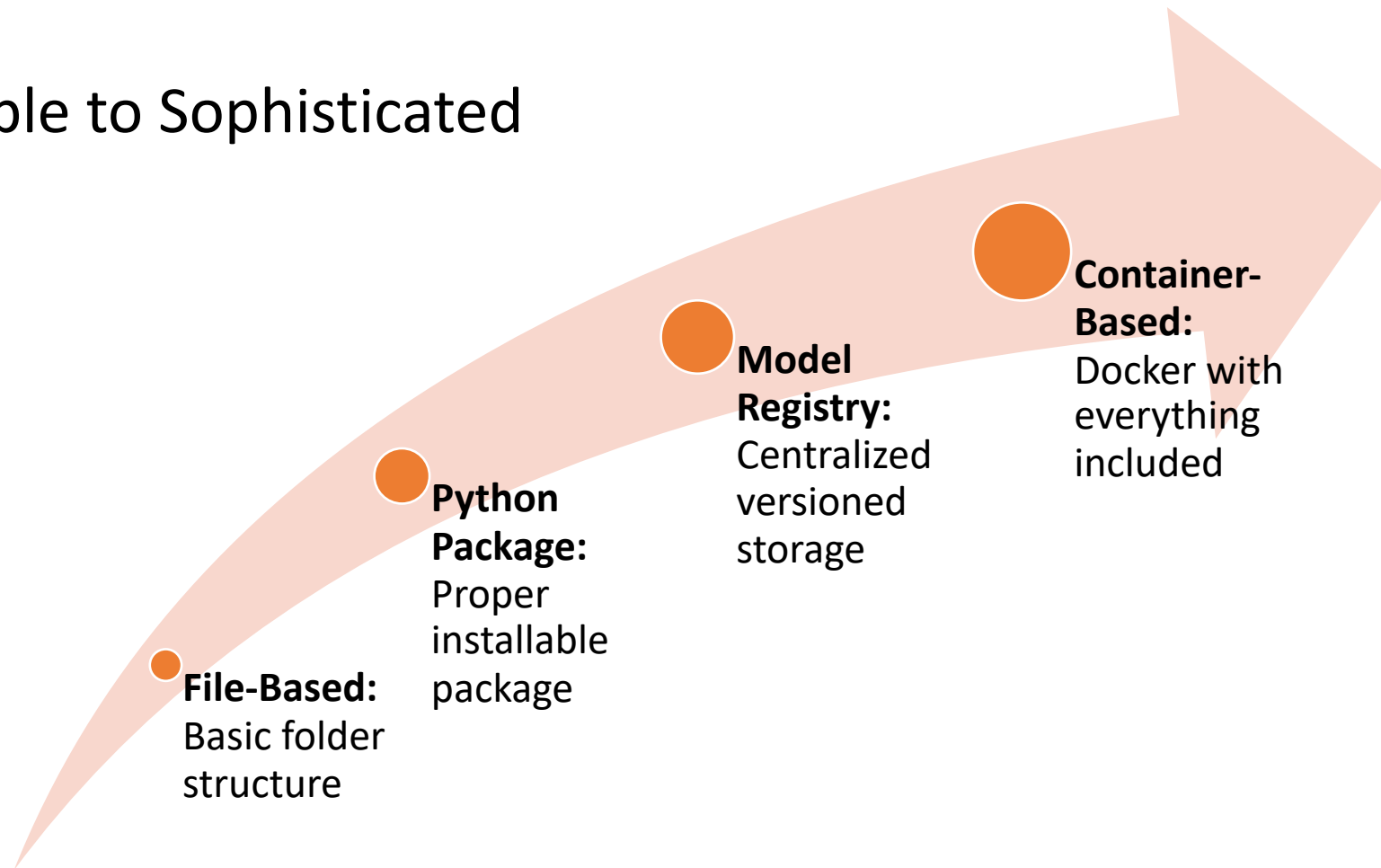
Beyond Just the Model File

What to Package:

- ✓ Trained model
- ✓ Preprocessing code
- ✓ Dependencies & environment
- ✓ Configuration files
- ✓ Documentation & metadata

Packaging Approaches

From Simple to Sophisticated



VisionaryAI Packaging Decision

Real-World Example

- **Defect Detection Model Requirements:**

- Cross-platform compatibility (factory systems)
- Consistent deployment
- Regular updates
- Easy integration

- **Our solution:**

ONNX format

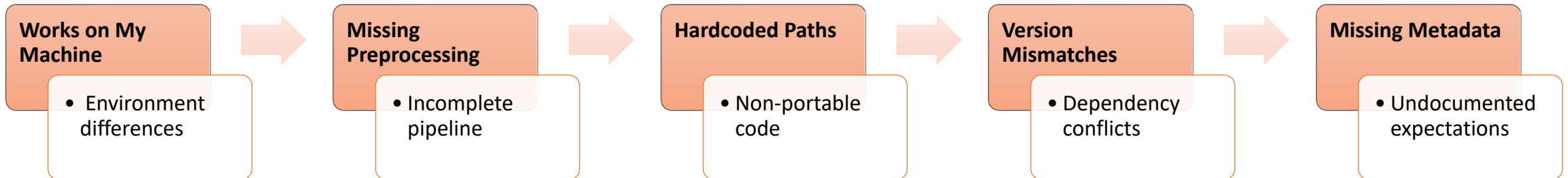
Docker containers

REST API

Common Packaging Pitfalls



Learn from These Mistakes:



Three Deployment Patterns

Where and How to Deploy

Batch Processing

Process data in chunks

Scheduled execution

Example: Nightly recommendations

Real-time/Streaming

On-demand predictions

API-based serving

Example: Support ticket classification

Edge Deployment

On-device inference

No network required

Example: Factory defect detection

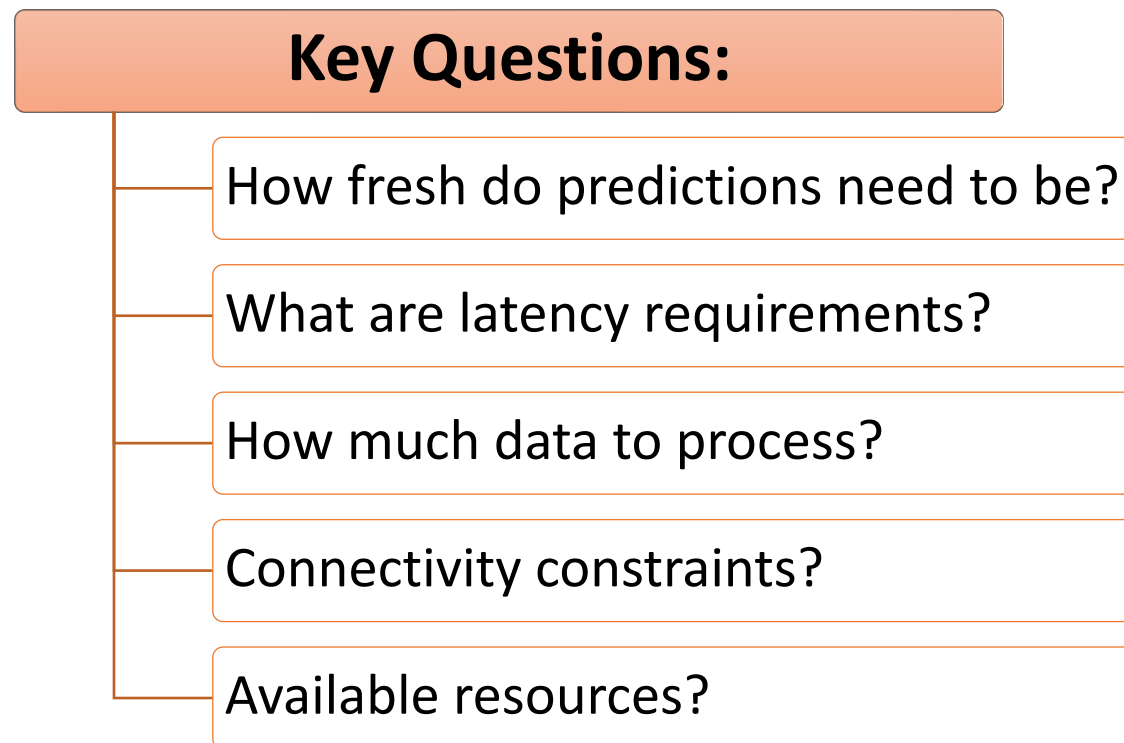
Batch vs Real-time vs Edge

Pattern Comparison

Aspect	Batch	Real-time	Edge
Latency	Minutes-Hours	Milliseconds	Microseconds
Freshness	Stale	Current	Current
Complexity	Low	Medium	High
Resources	Flexible	Predictable	Constrained
Connectivity	Required	Required	Optional

Decision Framework

Choosing the Right Pattern



API Design Principles

Making Models Accessible

RESTful Design:

- Clear, descriptive endpoints
- Appropriate HTTP methods
- Meaningful status codes
- Proper error handling

Example Endpoint: ``/api/v1/defects/detect``

VisionaryAI Defect Detection API

Well-Designed API Example

Request:

```
POST /api/v1/defects/detect
{
  "image": "base64_data",
  "factory_id": "plant_001",
  "production_line": "line_03"
}
```

Response:

```
{
  "prediction": {
    "has_defect": true,
    "confidence": 0.87
  },
  "metadata": {
    "model_version": "v2.1.0",
    "processing_time_ms": 45
  }
}
```

Error Handling & Versioning

Robust API Design

Error Handling:

- Informative error messages
- Appropriate HTTP status codes (200, 300, 400, 500)
- Graceful degradation

Versioning Strategy:

- URL versioning (`/api/v1/`)
- Backward compatibility
- Clear deprecation timelines

Scalability Fundamentals

Designing for Growth

Stateless Design:

- Each request self-contained

Caching Strategies:

- Reduce compute overhead

Load Balancing:

- Distribute traffic

Auto-scaling:

- Handle demand spikes

Performance Monitoring:

- Track key metrics

VisionaryAI Performance Targets

Real-World SLAs

- **Defect Detection:** < 100ms response time
- **Customer Support:** < 500ms classification
- **Recommendations:** < 200ms real-time suggestions
- **Uptime Goal:** 99.9% availability ($7*24*60*0.001=10.08$ min/week)
- **Error Rate:** < 0.1% false predictions

ML Security Threat Landscape

Unique Challenges for ML Applications

- **Input Poisoning:** Malicious inputs to fool models
- **Model Extraction:** Reverse-engineering through queries
- **Data Poisoning:** Corrupted feedback for retraining
- **Privacy Violations:** Leaking training data information

Input Security & Validation

First Line of Defense

- **Data Type Validation:** Ensure proper formats
- **Size Limits:** Prevent DoS attacks (1000 char max)
- **Content Filtering:** Screen for malicious patterns
- **Rate Limiting:** Protect against abuse & extraction
- **VisionaryAI Example:** Support tickets validation

Authentication & Authorization

Controlling Access to ML Services

- **API Key Management:** Unique keys per client/location
- **Role-Based Access:** Different capabilities by user type
- **Token Expiration:** Short-lived, renewable tokens
- **Audit Logging:** Track access and usage patterns

Data Privacy & Model Protection

Protecting Assets & Compliance

- **Data Encryption:** In-transit (HTTPS) and at-rest
- **Data Minimization:** Only collect what's needed
- **Anonymization:** Remove/replace identifying info
- **Model Obfuscation:** Prevent reverse engineering
- **Compliance:** GDPR, CCPA, HIPAA requirements

Secure Deployment Practices

Production Security Implementation

- **Container Security:** Minimal images, non-root users
- **Network Isolation:** Firewalls, VPNs, segmentation
- **Environment Separation:** Dev/staging/production isolation
- **Dependency Updates:** Balance security vs. stability
- **Security Monitoring:** Automated threat detection

VisionaryAI Security Example

Comprehensive Security Implementation

- **Customer Support Classifier Protection:**
 - API keys with 100 req/min rate limiting
 - Input validation & sanitization
 - 30-day data anonymization policy
 - Isolated container deployment
 - Anomaly pattern monitoring

Code Demo Preview

What We'll Build (With Security)

VisionaryAI Customer Support Classifier

Demo Steps:

- Model serialization (joblib)
- FastAPI service with security features
- Docker containerization (secure)
- Security testing and validation

Goal:

- Understand concepts, not production code

Session Summary

Key Takeaways

Serialization: Choose format based on requirements

Packaging: Consider deployment environment early

Patterns: Match deployment to business needs

APIs: Design for clarity and maintainability

Scalability: Plan for growth from the start

Security: Build in protection from day one

Next Session Preview

Production Operations & Monitoring

Coming Up:

- Monitoring deployed models
- Detecting model drift
- Logging and observability
- Incident response
- Performance optimization

From:

- Getting models to production

To:

- Keeping them healthy long-term

Questions Before Code Demo

- Pickle vs joblib - when to use which?
- Containers vs simpler packaging - trade-offs?
- Zero-downtime model updates - how?
- API overhead - performance impact?
- How do I implement security without complexity?
- Ready for hands-on demo?



دانستگار

THANK YOU

FOR YOUR ATTENTION!

